

CFGs ASIR Gestión de Bases de Datos Curso 25 – 26	UT 4. SQL. MODIFICACIÓN EN BASES DE DATOS
Actividad de Aula	Boletín 11 – Transacciones y acceso simultáneo a datos

Dataset de prácticas (DDL + DML) – “EmpresaMini”

Objetivo didáctico: ver qué pasa al borrar el “padre” cuando hay “hijos”, según la acción definida en la FK.

Aquí tienes un dataset de una empresa. Este boletín se basa en él. Más abajo tienes los ejercicios.

```

DROP DATABASE IF EXISTS empresa_mini;

CREATE DATABASE empresa_mini CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
USE empresa_mini;

-- Tabla PADRE
CREATE TABLE departamentos (
    id_depto INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(40) NOT NULL UNIQUE
) ENGINE=InnoDB;

-- HIJO 1: NO ACTION (comportamiento por defecto: RESTRINGE el borrado del padre)
CREATE TABLE empleados (
    id_emp INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(60) NOT NULL,
    id_depto INT NOT NULL,
    CONSTRAINT fk_emp_depto
        FOREIGN KEY (id_depto) REFERENCES departamentos(id_depto)
        ON DELETE NO ACTION
) ENGINE=InnoDB;

-- HIJO 2: CASCADE (al borrar el padre, se borran automáticamente los hijos)
CREATE TABLE proyectos (
    id_proy INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(60) NOT NULL,
    id_depto INT NOT NULL,
    CONSTRAINT fk_proy_depto

```

```

    FOREIGN KEY (id_depto) REFERENCES departamentos(id_depto)
    ON DELETE CASCADE
) ENGINE=InnoDB;

-- HIJO 3: SET NULL (al borrar el padre, el campo FK del hijo pasa a NULL)
-- OJO: id_depto debe permitir NULL para que SET NULL sea posible
CREATE TABLE incidencias (
    id_inc    INT PRIMARY KEY AUTO_INCREMENT,
    titulo    VARCHAR(80) NOT NULL,
    id_depto  INT NULL,
    CONSTRAINT fk_inc_depto
        FOREIGN KEY (id_depto) REFERENCES departamentos(id_depto)
        ON DELETE SET NULL
) ENGINE=InnoDB;

-- Datos iniciales
INSERT INTO departamentos (nombre) VALUES
('Sistemas'), ('Desarrollo'), ('Soporte');

INSERT INTO empleados (nombre, id_depto) VALUES
('Ana', 1),
('Carlos', 1),
('Lucía', 2);

INSERT INTO proyectos (nombre, id_depto) VALUES
('Migración a MariaDB', 1),
('App Móvil', 2),
('Portal Interno', 2);

INSERT INTO incidencias (titulo, id_depto) VALUES
('Impresora no imprime', 3),
('VPN caída', 1),
('Error al compilar', 2),
('Incidencia sin asignar', NULL);

```

Para usar un cliente SQL como Beekeeper crea un usuario, tal como este:

```

-- =====
-- Usuario de prácticas · empresa_mini
-- =====

-- Eliminar usuario si ya existe
DROP USER IF EXISTS 'empresa_act'@'localhost';

-- Crear usuario
CREATE USER 'empresa_act'@'localhost'
IDENTIFIED BY '1234';

-- Dar permisos SOLO sobre la base de datos empresa_mini
GRANT ALL PRIVILEGES ON empresa_mini.* TO 'empresa_act'@'localhost';

-- Aplicar cambios
FLUSH PRIVILEGES;

```

Instrucciones generales al alumnado

- Ejecuta las sentencias en el orden indicado.
- Observa el resultado final de los datos.
- Indica si los cambios **se confirman, se deshacen o quedan bloqueados**, y por qué.

BLOQUE 5 · TRANSACCIONES (10 ejercicios)

Ejercicio 1

Inicia una transacción.
 Actualiza el código del departamento SIST a SI.
 Cancela la transacción.

Ejercicio 2

Inicia una transacción.
 Actualiza el nombre del departamento con código DEV2 a Desarrollo Avanzado.
 Confirma la transacción.

Ejercicio 3

Inicia una transacción.
 Inserta un nuevo empleado en el departamento SI.
 Inserta otro empleado con un `cod_depto` inexistente.
 Intenta confirmar la transacción.

Ejercicio 4

Inicia una transacción.
 Actualiza dos proyectos distintos.
 Ejecuta ROLLBACK antes de confirmar.

Ejercicio 5

Inicia una transacción.

Actualiza el código de un departamento (por ejemplo SUP → SP).

Comprueba **antes del COMMIT** qué ven los datos desde la misma sesión.

Ejercicio 6

Inicia una transacción.

Actualiza varias filas de la tabla **empleados**.

Confirma la transacción y verifica el resultado.

Ejercicio 7

Inicia una transacción.

Elimina una incidencia concreta.

Cancela la transacción y comprueba si la incidencia sigue existiendo.

Ejercicio 8

Desactiva el modo autocommit.

Actualiza el nombre de un departamento.

Cierra la sesión sin hacer **COMMIT**.

Vuelve a conectarte y comprueba el estado de los datos.

Ejercicio 9

Activa de nuevo el modo autocommit.

Actualiza el nombre de un departamento.

Comprueba si es posible deshacer el cambio.

Ejercicio 10

Inicia una transacción.

Realiza un **DELETE** sobre un departamento que provoca efectos en cascada.

Cancela la transacción.

Comprueba si las filas eliminadas por **CASCADE** reaparecen.

BLOQUE 6 · ACCESO SIMULTÁNEO Y BLOQUEOS (10 ejercicios)

Nota: estos ejercicios se realizan **con dos conexiones abiertas** a la base de datos (por ejemplo, dos pestañas en Beekeeper).

Ejercicio 11

Desde la **sesión A**:

- Inicia una transacción.
- Actualiza un empleado concreto.

Desde la **sesión B**:

- Intenta actualizar el mismo empleado.

Observa el comportamiento.

Ejercicio 12

Desde la **sesión A**:

- Inicia una transacción.
- Actualiza un departamento.

Desde la **sesión B**:

- Intenta eliminar ese mismo departamento.

Ejercicio 13

Desde la **sesión A**:

- Inicia una transacción.
- Actualiza una fila de **proyectos**.

Desde la **sesión B**:

- Intenta hacer un **SELECT** sobre esa fila.

Ejercicio 14

Desde la **sesión A**:

- Inicia una transacción.
- Actualiza varias filas de **empleados**.

Desde la **sesión B**:

- Intenta actualizar un empleado distinto.

Ejercicio 15

Desde la **sesión A**:

- Inicia una transacción.
- Actualiza una incidencia.

Desde la **sesión B**:

- Intenta eliminar esa incidencia.

Ejercicio 16

Desde la **sesión A**:

- Inicia una transacción.
- Actualiza el código de un departamento.

Desde la **sesión B**:

- Intenta insertar un empleado con ese código de departamento.

Ejercicio 17

Desde la **sesión A**:

- Inicia una transacción.
- Ejecuta un **SELECT** sobre la tabla **empleados**.

Desde la **sesión B**:

- Actualiza un empleado distinto.

Observa si el **SELECT** bloquea o no.

Ejercicio 18

Desde la **sesión A**:

- Inicia una transacción.
- Actualiza una fila y **no hagas COMMIT**.

Desde la **sesión B**:

- Intenta leer esa fila y luego actualizarla.

Ejercicio 19

Desde la **sesión A**:

- Inicia una transacción.
- Actualiza un departamento.

Desde la **sesión B**:

- Intenta actualizar un proyecto del mismo departamento.

Ejercicio 20

Ejercicio final comparativo:

Realiza un mismo **UPDATE**:

- una vez con autocommit activado
- otra vez dentro de una transacción sin confirmar

Compara el comportamiento frente a otra sesión concurrente.